

Inventory Reorder Alert System

Python Intern Technical Assessment | Complete Submission Documentation

1. Assessment Summary & Approach

This submission implements an automated inventory alert backend job designed for daily warehouse stock scanning.

Core Design & Highlights:

- **File Parsing:** Utilizes `csv.DictReader` to extract row data into standard Python dictionary objects.
- **Priority Threshold Logic:** Items below threshold are automatically flagged as CRITICAL (stock $\leq$ 25% of threshold) or LOW (stock < threshold).
- **Reorder Calculation:** Suggests explicit restock quantities required to bring stock levels back up to 80% of max capacity.
- **Robust Edge-Case Handling:** Handled missing quantities (defaulting to 0 with log warnings), blank threshold fields, and bad rows gracefully without crashing.
- **Multiple Reporting Channels:** Formatted console outputs, simulated email bodies, exported `restock_report.csv`, and an interactive Streamlit UI.

Submission Links

GitHub Repository: https://github.com/vrhrs/fobe_skills

Core Backend Script: `inventory_alert.py`

Streamlit App Script: `app.py`

2. Backend Script (inventory_alert.py)

```
import csv
import os
import sys
from datetime import datetime

if sys.stdout.encoding and sys.stdout.encoding.lower() != "utf-8":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")

INPUT_FILE = "stock.csv"
OUTPUT_FILE = "restock_report.csv"
ALERT_EMAIL = "warehouse-manager@company.com"
SENDER_EMAIL = "alerts@inventory-system.com"

CRITICAL_PCT = 0.25 # below 25% of threshold = critical
HEALTHY_PCT = 0.80 # reorder target = 80% of max capacity

def load_stock(filepath):
    records = []
    script_dir = os.path.dirname(os.path.abspath(__file__))
    full_path = os.path.join(script_dir, filepath)

    if not os.path.exists(full_path):
        print(f"File not found: {full_path}")
        return records

    with open(full_path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row_num, row in enumerate(reader, start=2):
            try:
                row = {k: v.strip() for k, v in row.items() if k}

                if not row.get("item_name"):
                    print(f"Row {row_num}: missing item_name, skipped.")
                    continue

                qty_raw = row.get("current_quantity", "").strip()
                threshold_raw = row.get("reorder_threshold", "").strip()
                capacity_raw = row.get("max_capacity", "").strip()

                if qty_raw == "":
                    print(f"Row {row_num} ({row['item_name']}): no quantity value, defaulting to 0.")
                    current_qty = 0
                else:
                    current_qty = int(float(qty_raw))

                if threshold_raw == "":
                    print(f"Row {row_num} ({row['item_name']}): no threshold, skipped.")
                    continue
                threshold = int(float(threshold_raw))

                max_cap = int(float(capacity_raw)) if capacity_raw else threshold * 4

                records.append({
                    "sku": row.get("sku", "N/A"),
                    "item_name": row["item_name"],
                    "category": row.get("category", "General"),
                    "current_qty": current_qty,
                    "threshold": threshold,
                    "max_capacity": max_cap,
                    "unit": row.get("unit", "unit"),
                    "supplier": row.get("supplier", "Unknown"),
                })
            except:
```

```

        except (ValueError, KeyError) as e:
            print(f"Row {row_num}: could not parse row - {e}. Skipped.")

    return records

def classify_item(item):
    qty    = item["current_qty"]
    thresh = item["threshold"]
    cap    = item["max_capacity"]

    if qty >= thresh:
        return None

    if qty <= thresh * CRITICAL_PCT:
        priority = "CRITICAL"
    else:
        priority = "LOW"

    healthy_target = int(cap * HEALTHY_PCT)
    reorder_qty    = max(0, healthy_target - qty)

    return {
        **item,
        "priority":    priority,
        "reorder_qty": reorder_qty,
        "shortage":    thresh - qty,
    }

def build_restock_list(records):
    flagged = []
    for item in records:
        result = classify_item(item)
        if result:
            flagged.append(result)

    flagged.sort(key=lambda x: (x["priority"] != "CRITICAL", -x["shortage"]))
    return flagged

def print_console_report(flagged, total):
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    width = 72

    print("\n" + "=" * width)
    print("  INVENTORY REORDER REPORT")
    print(f"  Generated : {now}")
    print(f"  Scanned   : {total}    |   Flagged : {len(flagged)}")
    print("=" * width)

    if not flagged:
        print("\n  All stock levels are within acceptable ranges.\n")
        print("=" * width + "\n")
        return

    critical_items = [i for i in flagged if i["priority"] == "CRITICAL"]
    low_items      = [i for i in flagged if i["priority"] == "LOW"]

    def print_section(items, label):
        if not items:
            return
        print(f"\n  {label} ({len(items)} item{'s' if len(items) > 1 else ''})")
        print("    " + "-" * (width - 2))
        for item in items:
            print(f"      [{item['sku']}]  {item['item_name']}")

```

```

        print(f"    Category : {item['category']}")
        print(f"    Stock    : {item['current_qty']} {item['unit']} "
              f"(threshold: {item['threshold']}, max: {item['max_capacity']})")
        print(f"    Supplier : {item['supplier']}")
        print(f"    Shortage : {item['shortage']} {item['unit']} below threshold")
        print(f"    Reorder  : {item['reorder_qty']} {item['unit']} "
              f"(to reach {int(item['max_capacity'] * HEALTHY_PCT)} / 80% of capacity)")
        print()

    print_section(critical_items, "CRITICAL - Immediate action required")
    print_section(low_items, "LOW - Restock soon")

    print("=" * width)
    print(f"  Report saved to: {OUTPUT_FILE}")
    print("=" * width + "\n")

def simulate_email_alert(flagged):
    now = datetime.now().strftime("%A, %d %B %Y at %H:%M")
    critical_count = sum(1 for i in flagged if i["priority"] == "CRITICAL")

    low_count = sum(1 for i in flagged if i["priority"] == "LOW")

    subject = (
        f"[RESTOCK ALERT] {critical_count} Critical + {low_count} Low items "
        f"need attention - {datetime.now().strftime('%d %b %Y')}"
    )

    print("\n--- SIMULATED EMAIL ALERT ---\n")

    print(f"FROM    : {SENDER_EMAIL}")
    print(f"TO      : {ALERT_EMAIL}")
    print(f"SUBJECT : {subject}")
    print(f"DATE    : {now}")
    print()
    print("-" * 60)
    print("Dear Warehouse Manager,")
    print()
    print("Here is today's inventory scan report.")
    print(f"As of {now}, these items need to be restocked:")
    print()

    for item in flagged:
        priority_label = f"[{item['priority']}]"
        print(
            f"  {priority_label:<12} "
            f"{item['item_name']:<30} "
            f"Stock: {item['current_qty']:>4} / {item['threshold']} "
            f"-> Order: {item['reorder_qty']} {item['unit']}"
        )

    print()
    print("Please contact suppliers to place the necessary orders.")
    print()
    print("-" * 60)
    print("Inventory Alert System | Do not reply to this email.")
    print("-" * 60)
    print()

def export_csv_report(flagged):
    script_dir = os.path.dirname(os.path.abspath(__file__))
    output_path = os.path.join(script_dir, OUTPUT_FILE)

    fieldnames = [
        "priority", "sku", "item_name", "category",

```

```

        "current_qty", "reorder_threshold", "max_capacity",
        "shortage", "reorder_qty", "unit", "supplier",
    ]

    with open(output_path, "w", newline="", encoding="utf-8") as f:

        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        for item in flagged:
            writer.writerow({
                "priority": item["priority"],
                "sku": item["sku"],
                "item_name": item["item_name"],
                "category": item["category"],
                "current_qty": item["current_qty"],
                "reorder_threshold": item["threshold"],
                "max_capacity": item["max_capacity"],
                "shortage": item["shortage"],
                "reorder_qty": item["reorder_qty"],
                "unit": item["unit"],
                "supplier": item["supplier"],
            })

    print(f"Restock report saved to: {output_path}")

def print_reflection():
    print("\n" + "-" * 72)
    print(" REFLECTION NOTE")
    print("-" * 72)
    print(
        "With more time, there are a few things I'd want to improve here.\n"
        "\n"
        " 1. Scheduling - right now you have to run this manually. I'd set\n"
        "    it up as a cron job or Windows Task Scheduler so it runs every\n"
        "    morning automatically and the team always has a fresh report.\n"
        "\n"
        " 2. Supplier integration - instead of just flagging items, the script\n"
        "    could connect to supplier APIs and raise a purchase order directly\n"
        "    for anything in the critical tier.\n"
        "\n"
        " 3. Trend tracking - logging stock levels daily to a simple SQLite\n"
        "    database would let you spot which items drain fastest and adjust\n"
        "    thresholds based on actual usage rather than guesswork.\n"
        "\n"
        " 4. Real alerts - replacing the printed email simulation with smtplib\n"
        "    or a Slack webhook so the right people are notified immediately.\n"
        "\n"
        " 5. A basic dashboard - a lightweight Flask page where managers can\n"
        "    check stock health without needing to read a CSV or run anything.\n"
    )
    print("-" * 72 + "\n")

def main():
    print("\nLoading stock data from:", INPUT_FILE)

    records = load_stock(INPUT_FILE)

    if not records:
        print("No valid records found. Exiting.")
        return

    print(f"{len(records)} items loaded.\n")

    flagged = build_restock_list(records)

```

```

print_console_report(flagged, len(records))

if flagged:
    simulate_email_alert(flagged)
    export_csv_report(flagged)

print_reflection()

if __name__ == "__main__":
    main()

```

3. Streamlit Application Code (app.py)

```

import streamlit as st
import pandas as pd
import csv
import io
import os
from datetime import datetime

st.set_page_config(
    page_title="Inventory Reorder Alert System",
    layout="wide"
)

CRITICAL_PCT = 0.25
HEALTHY_PCT = 0.80

ALERT_EMAIL = "warehouse-manager@company.com"
SENDER_EMAIL = "alerts@inventory-system.com"

def load_stock_from_string(content):
    records = []
    warnings = []
    reader = csv.DictReader(io.StringIO(content))

    for row_num, row in enumerate(reader, start=2):
        try:
            row = {k: v.strip() for k, v in row.items() if k}

            if not row.get("item_name"):
                warnings.append(f"Row {row_num}: missing item_name, skipped.")
                continue

            qty_raw = row.get("current_quantity", "").strip()
            threshold_raw = row.get("reorder_threshold", "").strip()
            capacity_raw = row.get("max_capacity", "").strip()

            if qty_raw == "":
                warnings.append(f"Row {row_num} ({row['item_name']}): no quantity, defaulted to 0.")
                current_qty = 0
            else:
                current_qty = int(float(qty_raw))

            if threshold_raw == "":
                warnings.append(f"Row {row_num} ({row['item_name']}): no threshold, skipped.")
                continue
            threshold = int(float(threshold_raw))

            max_cap = int(float(capacity_raw)) if capacity_raw else threshold * 4

            records.append({

```

```

        "sku":          row.get("sku", "N/A"),
        "item_name":    row["item_name"],
        "category":     row.get("category", "General"),
        "current_qty":  current_qty,
        "threshold":     threshold,
        "max_capacity":  max_cap,
        "unit":         row.get("unit", "unit"),
        "supplier":      row.get("supplier", "Unknown"),
    })

    except (ValueError, KeyError) as e:
        warnings.append(f"Row {row_num}: could not parse - {e}. Skipped.")

    return records, warnings

def classify_item(item):
    qty    = item["current_qty"]
    thresh = item["threshold"]
    cap    = item["max_capacity"]

    if qty >= thresh:
        return None

    priority = "CRITICAL" if qty <= thresh * CRITICAL_PCT else "LOW"

    healthy_target = int(cap * HEALTHY_PCT)
    reorder_qty    = max(0, healthy_target - qty)

    return {
        **item,
        "priority":    priority,
        "reorder_qty": reorder_qty,
        "shortage":    thresh - qty,
    }

def build_restock_list(records):
    flagged = [classify_item(i) for i in records]
    flagged = [i for i in flagged if i is not None]
    flagged.sort(key=lambda x: (x["priority"] != "CRITICAL", -x["shortage"]))
    return flagged

def build_email_text(flagged):
    now = datetime.now().strftime("%A, %d %B %Y at %H:%M")
    critical_count = sum(1 for i in flagged if i["priority"] == "CRITICAL")
    low_count      = sum(1 for i in flagged if i["priority"] == "LOW")

    subject = (
        f"[RESTOCK ALERT] {critical_count} Critical + {low_count} Low items "
        f"need attention - {datetime.now().strftime('%d %b %Y')}"
    )

    lines = [
        f"FROM      : {SENDER_EMAIL}",
        f"TO        : {ALERT_EMAIL}",
        f"SUBJECT    : {subject}",
        f"DATE       : {now}",
        "",
        "-" * 60,
        "Dear Warehouse Manager,",
        "",
        "Here is today's inventory scan report.",
        f"As of {now}, these items need to be restocked:",
    ]

```

```

    """,
]

for item in flagged:
    label = f"[{item['priority']}]"
    lines.append(
        f" {label:<12} "
        f"{item['item_name']:<30} "
        f"Stock: {item['current_qty']:>4} / {item['threshold']} "
        f"-> Order: {item['reorder_qty']} {item['unit']}"
    )

lines += [
    "",
    "Please contact suppliers to place the necessary orders.",
    "",
    "-" * 60,
    "Inventory Alert System | Do not reply to this email.",
    "-" * 60,
]

return "\n".join(lines)

def to_csv_bytes(flagged):
    output = io.StringIO()
    fieldnames = [
        "priority", "sku", "item_name", "category",
        "current_qty", "reorder_threshold", "max_capacity",
        "shortage", "reorder_qty", "unit", "supplier",
    ]
    writer = csv.DictWriter(output, fieldnames=fieldnames)
    writer.writeheader()
    for item in flagged:
        writer.writerow({
            "priority": item["priority"],
            "sku": item["sku"],
            "item_name": item["item_name"],
            "category": item["category"],
            "current_qty": item["current_qty"],
            "reorder_threshold": item["threshold"],
            "max_capacity": item["max_capacity"],
            "shortage": item["shortage"],
            "reorder_qty": item["reorder_qty"],
            "unit": item["unit"],
            "supplier": item["supplier"],
        })
    return output.getvalue().encode("utf-8")

script_dir = os.path.dirname(os.path.abspath(__file__))
sample_path = os.path.join(script_dir, "stock.csv")
sample_csv_text = ""
if os.path.exists(sample_path):
    with open(sample_path, "r", encoding="utf-8") as f:
        sample_csv_text = f.read()

st.title("Inventory Reorder Alert System")
st.caption("Upload your stock CSV or use sample data to scan for items needing restock.")

st.divider()

col_a, col_b = st.columns([1, 1])

with col_a:

```



```

    if sample_csv_text:
        st.download_button(
            label="Download Sample stock.csv",
            data=sample_csv_text,
            file_name="stock.csv",
            mime="text/csv",
            help="Download the sample stock CSV file to test the upload feature."
        )

with col_b:
    use_sample = st.button("Use Sample Stock Data", help="Instantly run the scan using built-in sample data.")

uploaded = st.file_uploader("Or upload your own stock CSV file", type=["csv"])

content_to_parse = None

if uploaded is not None:
    content_to_parse = uploaded.read().decode("utf-8")
elif use_sample or st.session_state.get("used_sample", False):
    st.session_state["used_sample"] = True

    content_to_parse = sample_csv_text

if content_to_parse is None:
    st.info("Tip: Click 'Download Sample stock.csv' to get a test file, or click 'Use Sample Stock Data' to test immediately.")
    st.stop()

records, warnings = load_stock_from_string(content_to_parse)

if warnings:
    with st.expander(f"Parse warnings ({len(warnings)})"):
        for w in warnings:
            st.warning(w)

if not records:
    st.error("No valid records found in the file.")
    st.stop()

flagged = build_restock_list(records)

critical_items = [i for i in flagged if i["priority"] == "CRITICAL"]
low_items      = [i for i in flagged if i["priority"] == "LOW"]
ok_count       = len(records) - len(flagged)

c1, c2, c3, c4 = st.columns(4)
c1.metric("Total Items", len(records))
c2.metric("OK", ok_count)
c3.metric("Low Stock", len(low_items))
c4.metric("Critical", len(critical_items), delta=f"-{len(critical_items)}" if critical_items else None, delta_color="normal")

st.divider()

if not flagged:
    st.success("All stock levels are within acceptable ranges.")
    st.stop()

st.subheader("Items Needing Restock")

df = pd.DataFrame([
    "Priority":    i["priority"],
    "SKU":        i["sku"],
    "Item":       i["item_name"],
    "Category":   i["category"],
    "Stock":      i["current_qty"],
    "Threshold":  i["threshold"],
    "Max Cap.":   i["max_capacity"],

```

```

        "Shortage": i["shortage"],
        "Reorder Qty": i["reorder_qty"],
        "Unit": i["unit"],
        "Supplier": i["supplier"],
    } for i in flagged])

def highlight_priority(row):
    if row["Priority"] == "CRITICAL":
        return ["background-color: #fde8e8; color: #c0392b; font-weight: 600"] * len(row)
    elif row["Priority"] == "LOW":
        return ["background-color: #fef9e7; color: #b7770d; font-weight: 600"] * len(row)
    return [""] * len(row)

```

```

styled = df.style.apply(highlight_priority, axis=1)
st.dataframe(styled, use_container_width=True, hide_index=True)

```

```

st.download_button(
    label="Export restock_report.csv",
    data=to_csv_bytes(flagged),
    file_name="restock_report.csv",
    mime="text/csv",
)

```

```
st.divider()
```

```

st.subheader("Simulated Email Alert")
email_text = build_email_text(flagged)
st.code(email_text, language=None)

```

```
st.divider()
```

```

with st.expander("Reflection note"):
    st.markdown("""
With more time, there are a few things I'd want to improve here.

```

1. **Scheduling** – right now you have to run this manually. I'd set it up as a cron job or Windows Task Scheduler so
 2. **Supplier integration** – instead of just flagging items, the script could connect to supplier APIs and raise a pu
 3. **Trend tracking** – logging stock levels daily to a SQLite database would let you spot which items drain fastest a
 4. **Real alerts** – replacing the printed email simulation with smtplib or a Slack webhook so the right people are no
 5. **Better dashboard** – this Streamlit app is a start, but with more time I'd add historical charts and per-supplier
- ```

""")

```

## 4. Input Stock Data (stock.csv)

| sku      | item_name               | category   | current_quantity | reorder_threshold | max_capacity    | unit | supplier |
|----------|-------------------------|------------|------------------|-------------------|-----------------|------|----------|
| SKU-1042 | Cotton Fabric Roll      | Textiles   | 18,60,300        | roll              | TextilePro Ltd. |      |          |
| SKU-2087 | Packing Tape            | Packaging  | 240,30,500       | roll              | PackRight Inc.  |      |          |
| SKU-3311 | Steel Bolts (M6)        | Hardware   | 42,80,400        | pack              | BoltMaster Co.  |      |          |
| SKU-4456 | Cardboard Boxes (L)     | Packaging  | 9,50,250         | unit              | PackRight Inc.  |      |          |
| SKU-5502 | Bubble Wrap Roll        | Packaging  | 130,40,200       | roll              | PackRight Inc.  |      |          |
| SKU-6120 | Adhesive Labels         | Office     | 15,60,300        | sheet             | OfficeWorld     |      |          |
| SKU-7001 | Latex Gloves (Box)      | Safety     | 3,50,200         | box               | SafeSupply Co.  |      |          |
| SKU-7002 | Surgical Masks          | Safety     | 0,100,500        | pack              | SafeSupply Co.  |      |          |
| SKU-7003 | Hand Sanitizer 500ml    | Hygiene    | 0,30,150         | bottle            | CleanFirst Ltd. |      |          |
| SKU-8010 | Stretch Wrap Film       | Packaging  | 6,20,100         | roll              | PackRight Inc.  |      |          |
| SKU-8011 | Safety Goggles          | Safety     | 8,20,100         | pair              | SafeSupply Co.  |      |          |
| SKU-8012 | First Aid Kit           | Safety     | 1,5,30           | kit               | SafeSupply Co.  |      |          |
| SKU-9001 | Printer Ink Cartridge   | Office     | 3,10,40          | cartridge         | OfficeWorld     |      |          |
| SKU-9002 | Cleaning Cloths         | Hygiene    | 50,40,200        | pack              | CleanFirst Ltd. |      |          |
| SKU-9003 | Floor Sweeper Bags      | Hygiene    | 0,12,60          | bag               | CleanFirst Ltd. |      |          |
| SKU-9004 | Heavy Duty Shelf Labels | Office     | ,,25,150         | sheet             | OfficeWorld     |      |          |
| SKU-9005 | Pallet Wrap             | Packaging  | 18,20,100        | roll              | PackRight Inc.  |      |          |
| SKU-9006 | Safety Vest             | Safety     | 11,15,60         | unit              | SafeSupply Co.  |      |          |
| SKU-9007 | Nitrogen Gas Canister   | Industrial | 2,5,20           | canister          | GasWorks Ltd.   |      |          |
| SKU-9008 | Emergency Exit Signs    | Safety     | 7,5,30           | unit              | SafeSupply Co.  |      |          |

## 5. Execution Output & Email Alert

Loading stock data from: stock.csv  
Row 17 (Heavy Duty Shelf Labels): no quantity value, defaulting to 0.  
20 items loaded.

```
=====
INVENTORY REORDER REPORT
Generated : 2026-07-26 21:50:04
Scanned : 20 | Flagged : 16
=====

CRITICAL - Immediate action required (8 items)

[SKU-7002] Surgical Masks
 Stock : 0 pack (threshold: 100, max: 500)
 Shortage : 100 pack below threshold
 Reorder : 400 pack (to reach 400 / 80% of capacity)

[SKU-7001] Latex Gloves (Box)
 Stock : 3 box (threshold: 50, max: 200)
 Shortage : 47 box below threshold
 Reorder : 157 box (to reach 160 / 80% of capacity)

--- SIMULATED EMAIL ALERT ---

FROM : alerts@inventory-system.com
TO : warehouse-manager@company.com
SUBJECT : [RESTOCK ALERT] 8 Critical + 8 Low items need attention
DATE : Sunday, 26 July 2026 at 21:50

Dear Warehouse Manager,

Here is today's inventory scan report.
As of Sunday, 26 July 2026 at 21:50, these items need to be restocked:

[CRITICAL] Surgical Masks Stock: 0 / 100 -> Order: 400 pack
[CRITICAL] Latex Gloves (Box) Stock: 3 / 50 -> Order: 157 box
[CRITICAL] Adhesive Labels Stock: 15 / 60 -> Order: 225 sheet
[CRITICAL] Cardboard Boxes (L) Stock: 9 / 50 -> Order: 191 unit
[LOW] Cotton Fabric Roll Stock: 18 / 60 -> Order: 222 roll
[LOW] Steel Bolts (M6) Stock: 42 / 80 -> Order: 278 pack

Please contact suppliers to place the necessary orders.

```

## 6. Reflection Note

With more time, there are five key improvements I would introduce:

- 1. **Automated Job Scheduling:** Configure cron jobs or Windows Task Scheduler to execute automated daily stock audits at 07:00.
- 2. **Supplier REST EDI APIs:** Auto-trigger draft purchase orders for items reaching critical threshold status.
- 3. **Historical Consumption Tracking:** Log daily stock snapshots into SQLite to calculate moving usage trends and dynamically adjust thresholds.
- 4. **Active Alerting:** Connect smtp lib or Slack webhooks for immediate notification delivery.
- 5. **Enhanced Dashboard:** Expand Streamlit UI with supplier-level breakdown charts and stock depletion trends.